

Kotlin & Jetpack Compose Developer Guide

Volume 4

Data Classes | Coroutines Deep Dive | LaunchedEffect
Dispatchers | Debugging | Sync vs Async Patterns

For Flutter/Dart developers learning Kotlin & Jetpack Compose

Chapter 1: Data Classes - Dart vs Kotlin

The Problem They Solve

When you have a class that only holds data, you need boilerplate: equality comparison, string representation, and copying with modified fields. Both Dart and Kotlin solve this, but in very different ways.

Dart - No Built-in Data Class

Dart has NO built-in data class keyword. A plain class gives you nothing for free:

```
class Product {
  final String id;
  final String name;
  final double price;

  Product({required this.id, required this.name, required this.price});
}

final a = Product(id: '1', name: 'Phone', price: 99.0);
final b = Product(id: '1', name: 'Phone', price: 99.0);

print(a == b);          // false <- compares references, not values!
print(a.toString());    // Instance of 'Product' <- useless
// a.copyWith(...)      // doesn't exist
```

Option 1 - Manual boilerplate in Dart:

```
class Product {
  final String id;
  final String name;
  final double price;

  const Product({required this.id, required this.name, required this.price});

  // You write all of this by hand:
  @override
  bool operator ==(Object other) =>
    other is Product && other.id == id &&
    other.name == name && other.price == price;

  @override
  int get hashCode => Object.hash(id, name, price);

  @override
  String toString() => 'Product(id: $id, name: $name, price: $price)';
}
```

Kotlin & Jetpack Compose Guide - Vol 4

```
Product copyWith({String? id, String? name, double? price}) =>
    Product(id: id ?? this.id, name: name ?? this.name, price: price ?? this.price);
}
```

Option 2 - freezed package (code generation):

```
@freezed
class Product with _$Product {
    const factory Product({
        required String id,
        required String name,
        required double price,
    }) = _Product;
}

// freezed generates: ==, hashCode, toString(), copyWith() automatically
```

Kotlin - data class is a Language Keyword

Kotlin has 'data class' built into the language. The compiler generates everything automatically with zero extra packages or code generation setup:

```
data class Product(
    val id: String,
    val name: String,
    val price: Double
)

// Everything works out of the box:
val a = Product(id = "1", name = "Phone", price = 99.0)
val b = Product(id = "1", name = "Phone", price = 99.0)

println(a == b)           // true  <- structural equality
println(a)                // Product(id=1, name=Phone, price=99.0)
val c = a.copy(price = 49.0) // built-in copy
println(c)                // Product(id=1, name=Phone, price=49.0)
```

What the Kotlin Compiler Generates Automatically

Feature	What it does
equals()	Compares all constructor properties by value
hashCode()	Based on all constructor properties
toString()	ClassName(prop1=val1, prop2=val2, ...)
copy(...)	Clone with optional field overrides
componentN()	Enables destructuring assignments

Side-by-side Comparison

Feature	Dart plain class	Dart freezed	Kotlin data class
== by value	No	Yes	Yes
toString()	No	Yes	Yes
copyWith/copy	No	Yes	Yes
Needs package	No	freezed	Nothing
Needs codegen	No	Yes	No
Language keyword	No	No	Yes

Gotcha: Only Constructor Properties Are Included

[!] data class only generates equals/hashCode for constructor properties. Properties defined in the class body are ignored.

```
data class Product(  
    val id: String,    // <- included in equals/hashCode  
    val name: String  // <- included in equals/hashCode  
) {  
    // body property -- NOT included in equals/hashCode  
    val label = "Item: $name"  
}
```

Combining data class with sealed class

data class cannot be abstract or sealed on its own, but you combine them like this - exactly as used in your ProductsState.kt:

```
sealed class ProductState {  
    data class Loaded(val products: List<Product>) : ProductState()  
    data class Error(val message: String) : ProductState()  
    object Loading : ProductState() // no data -> object, not data class  
}
```

Chapter 2: suspend fun vs Future<T> in Dart

The Direct Mapping

Dart	Kotlin
Future<void>	suspend fun foo(): Unit
Future<String>	suspend fun foo(): String
Future<Product>	suspend fun foo(): Product
async keyword on function	suspend keyword on function
await someFuture	just call the function directly

Dart - async/await

```
// Dart: declares it returns a future
Future<String> fetchProduct(String id) async {
    await Future.delayed(Duration(seconds: 1)); // simulate network
    return 'Phone';
}

// caller must also be async to use await
Future<void> loadData() async {
    final name = await fetchProduct('1'); // explicit await
    print(name);
}
```

Kotlin - suspend fun

```
// Kotlin: suspend = this function can be paused without blocking the thread
suspend fun fetchProduct(id: String): String {
    delay(1000) // no 'await' needed - suspend funs pause automatically
    return "Phone"
}

// caller must also be a suspend fun (or inside a coroutine)
suspend fun loadData() {
    val name = fetchProduct("1") // NO 'await' keyword - just call it
    println(name)
}
```

Key Differences

1. No await keyword in Kotlin

In Dart you must explicitly write 'await'. In Kotlin, calling a suspend fun automatically suspends - it pauses until it

completes. No keyword needed.

```
// Dart - explicit await required
final result = await fetchProduct('1');

// Kotlin - just call it, it suspends automatically
val result = fetchProduct("1")
```

2. Future is an object, suspend is not

In Dart, `Future<String>` is an actual object you can store, pass around, and chain with `.then()`. In Kotlin, 'suspend fun' is not an object - it can only be called from another suspend function or a coroutine scope.

```
// Dart - Future is a value you can hold and chain
final Future<String> myFuture = fetchProduct('1');
myFuture.then((result) => print(result));

// Kotlin - you cannot 'hold' a suspend fun call
// You need a coroutine scope to launch it
viewModelScope.launch {
    val result = fetchProduct("1") // runs here, inside coroutine
    println(result)
}
```

3. Where suspend functions can be called

Context	Dart async fun	Kotlin suspend fun
From normal function	Yes (returns Future, fire & forget)	No - compile error
From async/suspend	Yes	Yes
From coroutine scope	N/A	Yes (launch, async, viewModelScope)

[NOTE] The rule: you can only call a suspend fun from inside another suspend fun OR from inside a coroutine (launch {}, async {}, etc.). Trying to call it from a normal fun is a compile error.

Chapter 3: launch{} - Fire and Forget Pattern

What launch{} Does

launch{} starts a coroutine in the background and returns immediately. It does NOT block the current thread or wait for the coroutine to complete. This is the 'fire and forget' pattern.

```
fun doSomething() {
    // launch starts the coroutine but doesn't wait for it
    viewModelScope.launch {
        fetchProduct("1") // runs in background
    }

    // this executes IMMEDIATELY, without waiting for fetchProduct
    println("I run first!")
}
```

Dart Equivalent

```
void doSomething() {
    fetchProduct('1'); // no await - fire and forget
    print('I run first!');
}
```

[NOTE] Same idea in both languages - just don't await/block and the code continues immediately.

Why You Always Need launch{} in Kotlin

In Dart you can just drop the 'await' anywhere. In Kotlin, suspend functions MUST be called from a coroutine, so you always need launch{} as the wrapper when calling from a normal function.

```
// WRONG - won't compile
fun doSomething() {
    fetchProduct("1") // ERROR: suspend function can't be called from normal fun
    println("I run first!")
}

// CORRECT
fun doSomething() {
    viewModelScope.launch {
        fetchProduct("1") // fine - inside coroutine
    }
    println("I run first!") // runs immediately
}
```

Sequential Execution Inside a Coroutine

Kotlin & Jetpack Compose Guide - Vol 4

Inside a coroutine (`launch{} block`), lines execute sequentially - each line waits for the previous to finish, exactly like Dart's `await` chain.

```
// Kotlin
viewModelScope.launch {
    val order = orderRepository.placeOrder(items) // waits
    cartRepository.clearCart()                    // then this
    _state.value = OrderState.Completed(order)    // then this
}

// Dart equivalent
final order = await orderRepository.placeOrder(items);
await cartRepository.clearCart();
_state.value = OrderState.completed(order);
```

Parallel Execution - `async` + `awaitAll`

If you want multiple operations to run at the same time (parallel), use `async{} instead of launch{} and then awaitAll:`

```
// Kotlin - parallel
LaunchedEffect(Unit) {
    val job1 = async { viewModel.loadProducts() }
    val job2 = async { viewModel.loadCategories() }
    awaitAll(job1, job2) // wait for BOTH to finish
    // continues here after both complete
}

// Dart equivalent
await Future.wait([
    viewModel.loadProducts(),
    viewModel.loadCategories(),
]);
```

Pattern	Kotlin	Dart
Sequential	suspend fun calls inside <code>launch{}</code>	<code>await</code> one, then <code>await</code> next
Parallel	<code>async{} + awaitAll()</code>	<code>Future.wait([...])</code>
Fire & forget	<code>launch{} without awaiting</code>	call without <code>await</code>

Chapter 4: How launch{} Works Under the Hood

The Mental Model

launch{} does NOT necessarily create a new thread. Here is how to think about it:

Thread = a worker
Dispatcher = a manager assigning tasks to workers
Coroutine = a task that can be paused and resumed

[NOTE] Kotlin coroutines are NOT threads. They are lightweight tasks scheduled on a Dispatcher. Multiple coroutines can run on a single thread.

Dispatchers - The Key Concept

A Dispatcher decides which thread(s) the coroutine runs on. There are four built-in dispatchers:

```
// Runs on the MAIN thread (UI thread) - default for viewModelScope
viewModelScope.launch(Dispatchers.Main) {
    updateUI() // safe - main thread
}

// Runs on a background thread pool - for heavy CPU work
viewModelScope.launch(Dispatchers.Default) {
    heavyCalculation() // won't freeze UI
}

// Runs on a thread optimized for blocking I/O
viewModelScope.launch(Dispatchers.IO) {
    readFromDisk() // file reads, database, network blocking calls
    networkCall()
}

// Runs on whatever thread calls it (rarely used)
viewModelScope.launch(Dispatchers.Unconfined) {
    // starts on current thread
}
```

Dispatcher	Thread	Use for
Dispatchers.Main	UI/Main thread	UI updates, ViewModel default
Dispatchers.IO	IO thread pool	Network, file, database
Dispatchers.Default	CPU thread pool	Heavy computation, sorting
Dispatchers.Unconfined	Current thread	Rarely used

How the Main Thread Event Loop Works

Kotlin & Jetpack Compose Guide - Vol 4

`viewModelScope` uses `Dispatchers.Main` by default - your coroutine runs on the MAIN thread, not a new thread. The main thread has a message queue just like Dart's event loop:

Main thread event loop:

```
[ task1 ] [ task2 ] [ coroutine_chunk ] [ task3 ] [ coroutine_resume ] ...
```

Step by step when you call `launch{}:`

```
fun doSomething() {
    viewModelScope.launch {           // 1. schedules coroutine, returns immediately
        println("A")                  // 3. runs when thread is free
        delay(1000)                   // 4. suspends - releases thread
        println("C")                  // 5. resumes after 1 second
    }
    println("B")                      // 2. runs immediately
}

// Output:
// B
// A
// (1 second later)
// C
```

Comparison with Dart's Event Loop

Dart works almost identically - event loop + microtask queue:

```
void doSomething() async {
    Future(() => print('A')); // schedules on event loop
    print('B');              // runs immediately
}

// Output: B, then A
```

Concept	Dart	Kotlin
Default execution	Main thread event loop	Main thread (<code>Dispatchers.Main</code>)
Background thread	<code>compute()</code> / <code>Isolate</code>	<code>Dispatchers.IO</code> / <code>Dispatchers.Default</code>
Pause without blocking	<code>await</code> + event loop	<code>suspend</code> + dispatcher
Creates new thread?	No (by default)	No (by default)
Cooperative scheduling?	Yes	Yes

Switching Dispatchers Mid-Coroutine with `withContext`

`withContext` lets you switch the dispatcher inside a running coroutine. This is how you move heavy work off the main thread and safely come back:

```
viewModelScope.launch {
    // still on Main thread here

    val result = withContext(Dispatchers.IO) {
        // NOW on a background thread from the IO pool
    }
}
```

```
        networkCall() // blocking call - safe here, won't freeze UI
    }

    // automatically back on Main thread here
    _state.value = result // safe to update UI
}
```

[OK] `withContext` is the Kotlin equivalent of Dart's `compute()` for moving work off the main thread. You don't need to manually switch back - Kotlin does it automatically after the block.

Chapter 5: LaunchedEffect - Composable Side Effects

What LaunchedEffect Does

LaunchedEffect is a Composable that automatically creates a coroutine and runs the block inside it. It is lifecycle-aware and tied to composition.

```
LaunchedEffect(Unit) {  
    viewModel.loadProducts() // called once when screen appears  
}
```

This is roughly equivalent to:

```
val scope = rememberCoroutineScope()  
scope.launch {  
    viewModel.loadProducts()  
}
```

But LaunchedEffect is smarter - it is lifecycle-aware and automatically cancelled when the composable leaves composition.

What the Key (Unit) Means

The first argument to LaunchedEffect is the KEY. It controls WHEN the block re-runs:

```
LaunchedEffect(Unit) { } // runs ONCE when composable enters, never again  
LaunchedEffect(userId) { } // runs once, then re-runs every time userId changes  
LaunchedEffect(a, b) { } // re-runs when either a or b changes
```

[NOTE] Unit never changes, so the block runs exactly once. This is equivalent to initState() in Flutter or viewDidLoad in iOS.

The Lifecycle of LaunchedEffect

```
LaunchedEffect(Unit) {  
    viewModel.loadProducts() // runs when screen first appears  
}
```

Step by step:

1. Composable enters screen -> LaunchedEffect starts coroutine -> calls loadProducts()
2. Composable recomposes -> coroutine is NOT restarted
3. Composable leaves screen -> coroutine is AUTOMATICALLY cancelled

Why Not Call Functions Directly in the Composable Body?

[!] Composable functions recompile constantly (potentially 60 times per second). Calling a function directly in the body means it fires on every recompilation!

```
@Composable  
fun ProductsScreen(viewModel: ProductsViewModel) {  
    viewModel.loadProducts() // WRONG - runs on every recompilation!
```

Kotlin & Jetpack Compose Guide - Vol 4

```
// ...  
}  
  
@Composable  
fun ProductsScreen(viewModel: ProductsViewModel) {  
    LaunchedEffect(Unit) {  
        viewModel.loadProducts() // CORRECT - runs only once  
    }  
    // ...  
}
```

Calling Normal and Suspend Functions Inside LaunchedEffect

LaunchedEffect runs inside a coroutine, so you can call BOTH normal functions and suspend functions freely:

```
LaunchedEffect(Unit) {  
    viewModel.loadProducts()           // normal fun - works  
    viewModel.someOtherFun()           // normal fun - works  
    delay(1000)                        // suspend fun - works  
    someRepository.fetchData()         // suspend fun - works  
}
```

[OK] Normal functions can always be called from anywhere. Suspend functions can be called because LaunchedEffect already provides the coroutine context.

Flutter Equivalent Mapping

Kotlin	Flutter
LaunchedEffect(Unit) { }	initState()
LaunchedEffect(value) { }	didUpdateWidget() checking old vs new value
Block auto-cancelled on leave	dispose() cancelling a subscription

Chapter 6: Sync vs Async - Controlling Execution Order

The Three Scenarios

Inside `LaunchedEffect` (or any coroutine), you have full control over whether code runs sequentially or in parallel. Here are the three main patterns:

Scenario 1: Sequential (each line waits)

```
LaunchedEffect(Unit) {  
    viewModel.loadProducts()    // waits for this to finish  
    viewModel.loadCategories()  // then this  
    delay(1000)                 // then waits 1 second  
    viewModel.doSomething()     // then this  
}
```

```
// Dart equivalent:  
await viewModel.loadProducts();  
await viewModel.loadCategories();  
await Future.delayed(Duration(seconds: 1));  
await viewModel.doSomething();
```

Scenario 2: Fire and forget (don't wait)

```
LaunchedEffect(Unit) {  
    launch {                    // fire and forget - doesn't block  
        viewModel.loadProducts() // runs in background  
    }  
    println("I run first")      // runs immediately without waiting  
}
```

```
// Dart equivalent:  
unawaited(viewModel.loadProducts()); // or just: viewModel.loadProducts();  
print('I run first');
```

Scenario 3: Parallel (run multiple at the same time)

```
LaunchedEffect(Unit) {  
    val job1 = async { viewModel.loadProducts() }  
    val job2 = async { viewModel.loadCategories() }  
    awaitAll(job1, job2) // wait for both  
    println("both done")  
}
```

```
// Dart equivalent:  
await Future.wait([  
    viewModel.loadProducts(),  
    viewModel.loadCategories(),  
])
```

```
]);  
print('both done');
```

Real Project Example - OrderScreen Analysis

In your OrderScreen.kt, the LaunchedEffect demonstrates these concepts clearly. Here are all three versions with their behavior explained:

Current implementation (normal fun - fire and forget internally):

```
LaunchedEffect(Unit) {  
    viewModel.placeOrder()    // normal fun - fires viewModelScope.launch internally  
                               // and returns IMMEDIATELY  
    println("test message") // runs right away - does NOT wait for order to complete  
}  
  
// Execution order:  
// 1. placeOrder() called -> internally fires launch{} and returns immediately  
// 2. println runs right away  
// 3. order completes in background, state updates, UI recomposes
```

Alternative 1 - if placeOrder() were a suspend fun (sequential):

```
// suspend fun placeOrder() { ... } <- hypothetical  
  
LaunchedEffect(Unit) {  
    viewModel.placeOrder()    // suspend fun - LaunchedEffect AWAITS it automatically  
    println("test message") // runs ONLY after order is fully placed  
}  
  
// Execution order:  
// 1. placeOrder() starts and suspends the coroutine  
// 2. order completes  
// 3. println runs after
```

Alternative 2 - suspend fun + launch inside (fire and forget again):

```
// suspend fun placeOrder() { ... } <- hypothetical  
  
LaunchedEffect(Unit) {  
    launch {  
        viewModel.placeOrder()    // suspend fun - runs in background, NOT awaited  
    }  
    println("test message")    // runs IMMEDIATELY - does not wait for placeOrder  
}  
  
// Execution order:  
// 1. launch{} fires off placeOrder() in background and returns immediately  
// 2. println runs right away  
// 3. order completes later in background
```

Kotlin & Jetpack Compose Guide - Vol 4

[NOTE] Key insight: wrapping a suspend fun call in launch{} converts it from sequential (awaited) to fire-and-forget (background). This is the same concept as removing 'await' in Dart.

suspend fun in ViewModel - Which Approach is Better?

The ViewModel has two options for placeOrder. Here is why the current approach is preferred:

Approach	ViewModel	Composable caller	Recommended?
Current (normal fun)	fun placeOrder()	{ Button(onClick = {	Yes
Alternative (suspend fun)	suspend fun placeOrder() { ... }	scope.launch { viewModel.placeOrder() }	No

[OK] Keep ViewModel functions as normal fun{} with launch{} inside. The UI stays simple and dumb - it does not need to know about coroutines.

[!] Making ViewModel functions suspend pushes coroutine management to the UI layer. Now the Composable needs rememberCoroutineScope() and scope.launch{} - the UI becomes aware of async details it should not care about.

Chapter 7: Debugging - println vs Log in Android

Where Does println Output Go?

In Android, `println()` maps to `System.out.println()`. It does NOT print to a terminal console like in a regular Kotlin/JVM app. It prints to LOGCAT - Android Studio's log viewer.

```
LaunchedEffect(Unit) {
    println("test message") // appears in Logcat tagged as 'System.out'
                           // NOT in any terminal or console window
}
```

The Proper Way to Debug in Android

Use `android.util.Log` instead of `println`. It gives you tags, levels, and easy Logcat filtering:

```
import android.util.Log

LaunchedEffect(Unit) {
    Log.d("DEBUG", "screen opened")           // debug level
    Log.i("OrderScreen", "placing order")      // info level
    Log.e("OrderScreen", "something failed")    // error level - shows in red
    Log.w("OrderScreen", "cart is empty")      // warning level
}
```

Log Levels

Method	Level	Color in Logcat	Use for
<code>Log.v()</code>	VERBOSE	White	Very detailed, noisy logs
<code>Log.d()</code>	DEBUG	Blue	General debugging info
<code>Log.i()</code>	INFO	Green	Important events, milestones
<code>Log.w()</code>	WARN	Yellow	Potential issues, warnings
<code>Log.e()</code>	ERROR	Red	Errors, exceptions

How to Find Your Logs in Logcat

In Android Studio - bottom panel - Logcat tab. Filter options:

```
tag:DEBUG      <- only logs with your DEBUG tag
tag:System.out  <- println output
tag:OrderScreen <- only your OrderScreen logs
package:mine    <- only your app's logs (filters out system noise)
```

Dart vs Android Debugging Comparison

Dart/Flutter	Android/Kotlin	Where output appears
--------------	----------------	----------------------

Kotlin & Jetpack Compose Guide - Vol 4

<code>print('message')</code>	<code>Log.d('TAG', 'message')</code>	Logcat (Android Studio)
<code>debugPrint('message')</code>	<code>Log.d('TAG', 'message')</code>	Logcat (Android Studio)
<code>print()</code> in terminal app	<code>println()</code> in Logcat	<code>System.out</code> in Logcat

[NOTE] If your `println` is not showing up - check that you are looking at Logcat in Android Studio, not a terminal. Filter by 'System.out' if there is too much noise.

Chapter 8: Quick Reference - Everything in One Place

Coroutine Cheat Sheet

What you want	How to do it
Call suspend fun from normal fun	wrap in <code>viewModelScope.launch { }</code>
Fire and forget (don't wait)	<code>launch { }</code> without awaiting
Run two things in parallel	<code>async { }</code> + <code>awaitAll()</code>
Run once when screen opens	<code>LaunchedEffect(Unit) { }</code>
Re-run when value changes	<code>LaunchedEffect(value) { }</code>
Move work to background thread	<code>withContext(Dispatchers.IO) { }</code>
Heavy CPU computation	<code>withContext(Dispatchers.Default) { }</code>
Debug output in Android	<code>Log.d("TAG", "message")</code>

Dart to Kotlin Translation Table

Dart	Kotlin
<code>Future<String></code>	<code>suspend fun foo(): String</code>
<code>async function</code>	<code>suspend fun</code>
<code>await someFunction()</code>	<code>someFunction()</code> inside coroutine
<code>Future.wait([a(), b()])</code>	<code>async{a()} + async{b()} + awaitAll()</code>
<code>unawaited(someFunction())</code>	<code>launch { someFunction() }</code>
<code>initState()</code>	<code>LaunchedEffect(Unit) { }</code>
<code>compute()</code>	<code>withContext(Dispatchers.Default) { }</code>
<code>print()</code>	<code>Log.d("TAG", "") / println() -> Logcat</code>
<code>data class (via freezed)</code>	<code>data class (built-in keyword)</code>
<code>copyWith()</code>	<code>.copy()</code>

The ViewModel Pattern - Definitive Summary

```
@HiltViewModel
class OrderViewModel @Inject constructor(
    private val orderRepository: IOrderRepository,
) : ViewModel() {

    // State - private mutable, public read-only
    private val _state = MutableStateFlow<OrderState>(OrderState.Initial)
    val state: StateFlow<OrderState> = _state.asStateFlow()

    // Rule: ViewModel functions called from UI = normal fun + launch inside
    fun placeOrder() {
        viewModelScope.launch { // creates coroutine, returns immediately
```

```
        _state.value = OrderState.Loading
        try {
            val order = orderRepository.placeOrder() // awaited automatically
            _state.value = OrderState.Completed(order)
        } catch (e: Exception) {
            _state.value = OrderState.Error(e.message ?: "Unknown error")
        }
    }
}
```

```
@Composable
fun OrderScreen(viewModel: OrderViewModel = hiltViewModel()) {
    val state by viewModel.state.collectAsState()

    // LaunchedEffect - runs once, provides coroutine context
    LaunchedEffect(Unit) {
        viewModel.placeOrder() // normal fun - fine to call anywhere
    }

    // UI reacts to state automatically
    when (state) {
        is OrderState.Loading -> CircularProgressIndicator()
        is OrderState.Completed -> OrderSummary(state.order)
        is OrderState.Error -> ErrorMessage(state.message)
    }
}
```